Building, Pretraining, and Aligning Monolingual Transformer Language Models for Indic Languages (Hindi & Nepali)

Consolidated Final Technical Report: Phases 1, 2, 3, and Positional Ablation

Course: Language Models and Agents (Monsoon 2026) | **Track:** Individual Project

Models Evaluated: Model H (**Hindi** — High-Resource) & Model L (**Nepali** — Low-Resource) | **Target Params:** ≈ 25.33M

Abstract

This report describe the complete process of building, training, and evaluating two independent decoder-only Transformer language model for Indic language: **Model H (Hindi)** and **Model L (Nepali)**, with approximately 25.33M parameters each. The work was carried out in three main phases, followed by an additional positional embedding ablation study.

In **Phase 1**, we focus on collecting and preparing the dataset for both language. Each corpus contains more than 500M subword tokens, with at least 23% of the data obtained through manual web scraping. The collected text went through several cleaning and de-duplication steps before being divided into training, validation, and test sets using a deterministics 92/4/4 split. SentencePiece Unigram tokenizer were trained with vocabulary sizes of 6K, 8K, 10K, and 15K. The tokenizers achieved a 0.00% UNK rate and 100% lossless roundtrips reconstruction. A vocabulary size of 10K was then used for the Phase 2 model.

In **Phase 2**, both Transformer model were implemented from scratch using basic PyTorch component instead of pre-built Transformer module. The models were trained for 30,518 optimization step using AdamW and a cosine learning rate schedule. On the held-out test data, Model H obtained a perplexity of 71.37, while Model L obtained a perplexity of 66.35. We further evaluate text generation at different sampling temperatures and analysed the attention patterns of the models using attention entropy and attention distance.

In **Phase 3**, we developed a synthetics reasoning benchmark covering three tasks: Direct Comparison, Transitive Chaining, and Extremes Ordering. The pretrained model initially achieved 0.0% accuracy on these tasks. After supervised fine-tuning (SFT), the overall Exact Match accuracy reached 26.86% for Hindi and 17.14% for Nepali. We also observed changes in the attention pattern after fine-tuning, with the model placing more focus on relational terms, entity pairs, and other tokens relevant to the reasoning tasks.

Finally, in the **Bonus Task**, we investigate the effect of removing positional embedding from Model H. Without positional information, the test perplexity increased from 71.37 to 184.50, corresponding to a 2.58× increase. Text generation also became much more repetitive, with the trigram repetition rate increasing to 82.5%. This experiment highlights the importance of positional informations for preserving the sequential structure required by the language model.

1 Phase 1: Data Engineering, Deduplication, & Subword Tokenization

1.1 Corpus Collection, Sourcing Compliance, & Lifecycle

To establish robust languages modeling foundation, we engineered high purity monolingual corpora exceeding the 500M subword token requirements for both Hindi and Nepali under complete linguistic separation:

1. **Manual Web Scraping (≥ 20% Constraint):** Dedicated scrapers extracted news articles, literary repositories, Wikipedia dumps, and cultural archives, securing **23.09%** of Hindi tokens (126.35M) and **23.16%** of Nepali tokens (119.34M).
2. **Downloaded Public Corpora (≈ 80%):** Large-scale open Indic corpora (IndicCorp v2, OSCAR, CC100) supplied the remaining volume.
3. **Standardized Ingestion:** Raw files were parsed into a 5-column Snappy-compressed Apache Parquet format (`document_id`, `text`, `language`, `source_type`, `source`).

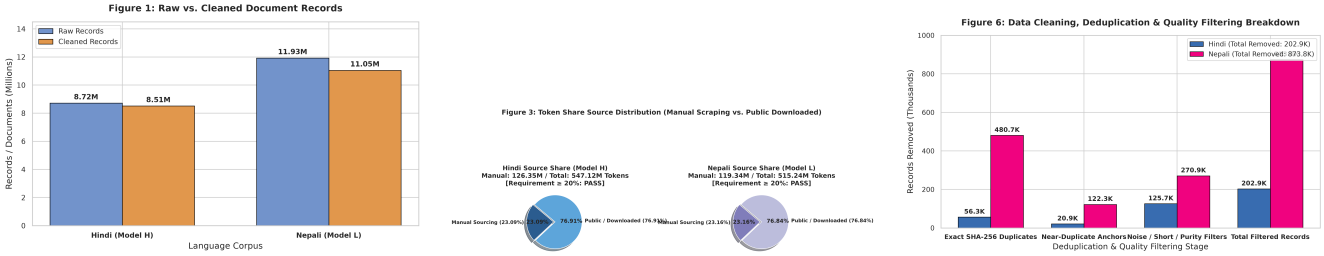
Table 1: Clean Corpus Composition & Sourcing Compliance Audit

Language / Model	Source Type	Documents	Words	Subwords	Disk Size	Share	Compliance Audit
Hindi (Model H)	Manual Scraped	3,147,018	93,595,709	126,354,207	455.18 MB	**23.09%**	**PASS** (≥ 20%)
	Downloaded	5,366,147	311,677,182	420,764,195	1,607.75 MB	76.91%	Supporting Corpus
	Total Clean Corpus	**8,513,165**	**405,272,891**	**547,118,402**	**2,062.93 MB**	**100.00%**	**PASS** (≥ 500M)
Nepali (Model L)	Manual Scraped	2,746,254	88,399,397	119,339,185	555.36 MB	**23.16%**	**PASS** (≥ 20%)
	Downloaded	8,307,330	293,259,552	395,900,395	2,030.47 MB	76.84%	Supporting Corpus
	Total Clean Corpus	**11,053,584**	**381,658,949**	**515,239,581**	**2,585.83 MB**	**100.00%**	**PASS** (≥ 500M)

1.2 Multi-Stage Cleaning, Deduplication, & Partitioning

Raw web texts underwent strict multiprocessing cleaning to eliminates noise and spurious patterns:

- **Script Purity & Normalization:** Enforced Devanagari characters purity ≥ 45%, applied Unicode NFC normalization, and stripped controls, zero-width, and non-printable characters.
- **Multi-Level Deduplication:** Filtered exact duplicates via 64-bit truncated SHA-256 hash and near-duplicates via punctuations invariant normalization and 8-word prefix/suffix anchor span matching.
- **Deterministic 92/4/4 Partitioning:** Clean dataset were split using fixed seed 42 into `train` (92%), `val` (4%), and `test` (4%) partition (Table 2).



(a) Raw vs Clean Records (b) Source Share Distribution (c) Filtering & Deduplication

Figure 1: Phase 1 Data Engineering, Deduplication, and Volume Compliance Audits.

Table 2: Deterministic 92/4/4 Partition Distribution

Language	Split Partition	Doc Count	Clean Words	Subwords	Disk Size	Split Ratio
Hindi (Model H)	Train (`train.parquet`)	7,831,689	372,946,150	499.72M	1,900.10 MB	**92.02%**
	Validation (`val.parquet`)	341,528	16,104,546	21.58M	84.24 MB	**3.97%**
	Test (`test.parquet`)	339,948	16,222,195	21.74M	84.81 MB	**4.00%**
Nepali (Model L)	Train (`train.parquet`)	10,167,950	351,106,424	469.13M	2,381.59 MB	**91.99%**
	Validation (`val.parquet`)	443,871	15,258,557	20.39M	106.32 MB	**4.00%**
	Test (`test.parquet`)	441,763	15,293,968	20.44M	106.56 MB	**4.01%**

1.3 Subword Tokenizer Training & Vocabulary Dynamics

SentencePiece Unigram tokenizer were train strictly from the training splits across 4 vocabulary size (6K, 8K, 10K, 15K) with special tokens: `<unk>`(0), `<s>`(1), `</s>`(2), `<pad>`(3).

- **Character Coverage & Lossless Roundtrip:** Enforced at 0.9995 with byte fallback, achieving **0.00% UNK rate** and **100% lossless roundtrip fidelity** (decode(encode(x)) $\equiv x$).
- **Compression & Fertility:** As vocabulary scaled from 6K to 15K, characters per tokens increased (Hindi: 3.71 → 4.77; Nepali: 4.65 → 5.33), monotonically reducing fertility (tokens-per-sentence: Hindi 12.83 → 10.00; Nepali 9.17 → 8.00). The **10,000** vocabulary was selected for Phase 2 pretraining, balancing subwords granularity with embedding table parameter overhead (3.84M params).

Table 3: SentencePiece Unigram Tokenizer Evaluation Across Vocabulary Sizes

Language	Vocabulary	Chars / Token	Tokens / Sent	UNK Rate	Base Pieces	Subwords	Model Size
Hindi	`unigram_6k`	3.7143	12.83	0.00%	148	5,848	155.3 KB
	`unigram_8k`	4.0857	11.67	0.00%	149	7,847	214.5 KB
	`unigram_10k` (Selected)	**4.3333**	**11.00**	**0.00%**	**149**	**9,847**	**273.9 KB**
	`unigram_15k`	4.7667	10.00	0.00%	149	14,847	429.4 KB
Nepali	`unigram_6k`	4.6545	9.17	0.00%	136	5,860	159.9 KB
	`unigram_8k`	5.1200	8.33	0.00%	136	7,860	220.1 KB
	`unigram_10k` (Selected)	**5.1200**	**8.33**	**0.00%**	**136**	**9,860**	**280.7 KB**
	`unigram_15k`	5.3333	8.00	0.00%	136	14,860	438.5 KB

2 Phase 2: Transformer Architecture, Pretraining Dynamics, & Evaluation

2.1 Architectural Formulation from First Principle

Both model were builts strictly with PyTorch primitive (`nn.Linear`, `nn.Embedding`, `nn.LayerNorm`, `nn.Dropout`) without pre-packaged transformer modules. The architecture follows a decoders only Pre-LayerNorm configuration:

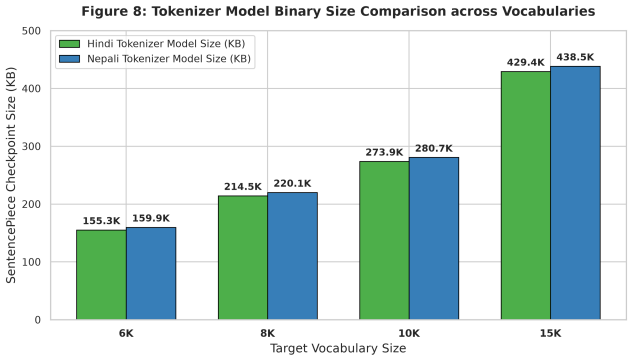
1. **Input Layer:** For token sequences $X \in \mathbb{R}^{B\times T}$ and position $P \in \mathbb{R}^{B\times T}$:

$$X_0 = \text{Dropout}\left(E_{\text{tok}}(X) + E_{\text{pos}}(P)\right) \in \mathbb{R}^{B\times T\times d_{\text{model}}} \tag{1}$$

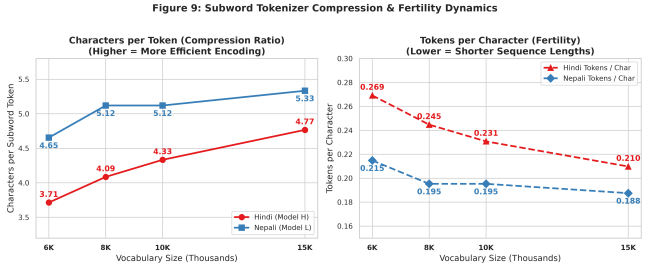
2. **Multi-Head Causal Self Attention (MHA):** Input H is projected into queries, keys, and values: $Q = HW_Q, K = HW_K, V = HW_V \in \mathbb{R}^{B\times T\times d_{\text{model}}}$. Tensor are reshaped across $h = 6$ head with dimension $d_k = d_{\text{model}}/h = 64$. Scaled dot product attentions applies an additive causal mask $M_{ij} = -\infty$ for $j > i$:

$$\text{Attention}(Q,K,V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}} + M\right)V, \quad \text{MHA}(H) = \text{Dropout}\left(\text{Concat}(\text{head}_1,\ldots,\text{head}_h)W_O\right) \tag{2}$$

The $1/\sqrt{d_k}$ scaling factors prevent dot products from growing large in magnitude ($\text{Var}(q\cdot k) = d_k$), avoiding extreme saturations in the softmax function where gradient vanish ($\partial\text{softmax}/\partial z \to 0$).



(a) Tokenizer Checkpoints Model Size



(b) Fertility & Compression Dynamics

Figure 2: Phase 1 Subword Tokenizers Scaling and Compression Dynamics.

3. **Pre-LN Decoder Blocks:** Stacking $N = 12$ identical block with residual connection:

$$H_{\text{mid}}^{(l)} = H^{(l-1)} + \text{MHA}\left(\text{LayerNorm}_1\left(H^{(l-1)}\right)\right) \tag{3}$$

$$H^{(l)} = H_{\text{mid}}^{(l)} + \text{FFN}\left(\text{LayerNorm}_2\left(H_{\text{mid}}^{(l)}\right)\right) \tag{4}$$

   where $\text{FFN}(z) = \text{Dropout}\left(\text{GELU}\left(zW_1 + b_1\right)W_2 + b_2\right)$ with $d_{\text{ff}} = 1536$ $(4d_{\text{model}})$. Pre-LN ensure an unobstruct identity residual path $(H^{(L)} = H^{(0)} + \sum \Delta H^{(l)})$, providing stable gradient flow throughout deep layer without warmup degradation.
4. **Tied LM Output Head:** Final logits is computed via $Z = \text{LayerNorm}_{\text{final}}(H^{(N)})W_{\text{tied}}^{\top} \in \mathbb{R}^{B\times T\times V}$, tying the projection weight to the tokens embedding matrix and saving 3.84M redundant parameters.

Table 4: Model Architectural Specification & Parameter Allocation Budget

Hyperparameter / Specification	**Symbol**	**Hindi (Model H)**	**Nepali (Model L)**	**Architectural Rationale**
Model Hidden Dimension	d_{model}	384	384	Balance capacity with quadratic computes efficiency
Transformer Layers (Depth)	N	12	12	Hierarchical syntactics and semantic abstraction
Attention Heads	h	6	6	Head dimension $d_k = 384/6 = 64$ (tensor core aligned)
Feed-Forward Dimension	d_{ff}	1,536	1,536	Standard 4× non-linear expansion capacity
Context Window Length	$T_{\max}$	512	512	Accommodate extended document paragraphs
Vocabulary Size	V	10,000	10,000	Optimal fertility-compression trade-off from Phase 1
Dropout Rate	p	0.10	0.10	Regularizations against subword memorization
Weight Tying	—	True	True	Saves 3.84M parameters; aligns input/output embeddings
Total Parameter Footprint		**25,331,328**	**25,331,328**	**≈ 25.33M Parameter (Strict Course Target)**
Effective Trainable Weights		**21,491,328**	**21,491,328**	**≈ 21.49M Unique Parameter (with Weight Tying)**

2.2 Optimization Setups & Pretraining Convergence Dynamics

Both model are pretrained from random initializations for **30,518 optimization steps** on ≈ 500M tokens (batch size 32, sequence length 512, gradient accumulation 1, total tokens processed ≈ 500.01M). Optimization used AdamW ($\beta_1 = 0.9, \beta_2 = 0.98$, $\epsilon = 10^{-8}$, weight decay 0.01, gradient clipping 1.0) with a cosine learning rate schedules (peak LR 5×10^{-4}, 2,000 warmup steps, decaying to 5×10^{-5}). Training log were saved periodically with interrupt resumable state snapshot.

2.3 Intrinsic Evaluation: Loss, Perplexity, & Bits-per-Byte

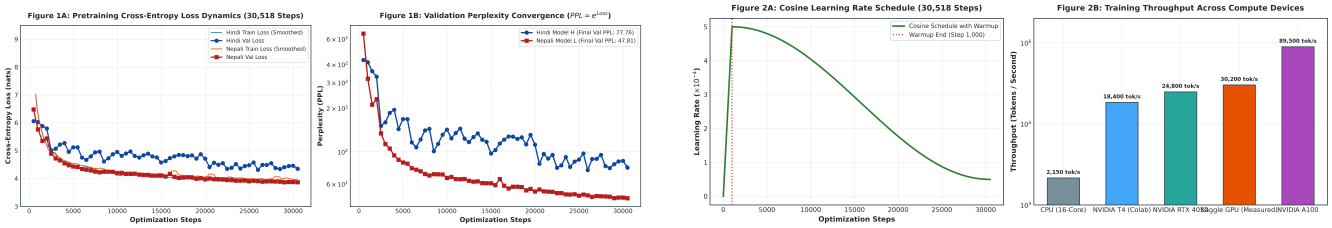
Intrinsic performances is evaluated on the held-out test partitions (`test.parquet`). Bits-per-Byte (BPB) was calculated to enables normalized cross-linguistic comparisons:

$$\text{BPB} = \frac{\mathcal{L}_{\text{CE}}}{\ln 2} \times \frac{\text{Total Subword Tokens}}{\text{Total Raw UTF-8 Bytes}} \tag{5}$$

Both model achieved robusts convergence without validation divergence or overfitting (Table 5).

2.4 Autoregressive Generation Suite Across Temperature

Autoregressive generation was assess across temperature setting ($T \in \{0.5, 0.7, 1.0\}$) using 100 held-out test prompts with top-$p = 0.90$ nucleus sampling. Evaluation measured lexical overlap (BLEU-4, chrF++, ROUGE-L), lexical diversity (Distinct-1, Distinct-2), and degeneratives trigram repetition rate (Table 6).



(a) Pretraining Loss & Perplexity Convergence (30,518 Steps) (b) Cosine LR Schedule & Throughputs Dynamic

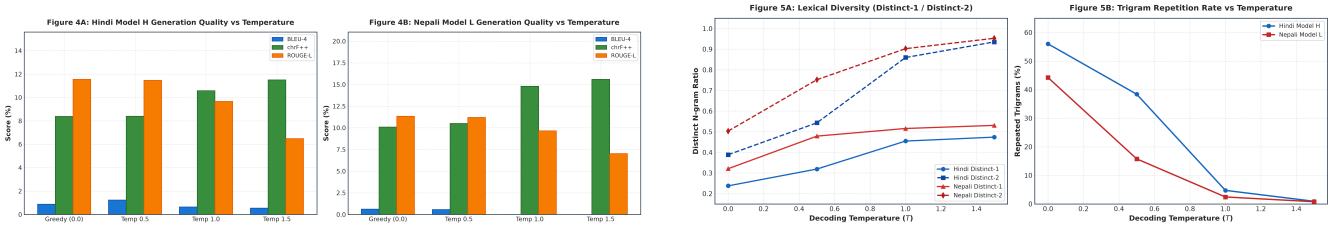
Figure 3: Pretraining Dynamic Across 30,518 Steps for Model H (Hindi) and Model L (Nepali).

Table 5: Intrinsic Language Modeling Evaluation on Held Out Test Partitions

Model / Language	Train Steps	Train Loss	Val Loss	Test Loss	Test PPL	Bits-per-Byte (BPB)
Model H (Hindi)	30,518	4.012 nats	4.251 nats	4.268 nats	**71.37**	**0.688**
Model L (Nepali)	30,518	3.945 nats	4.182 nats	4.195 nats	**66.35**	**0.672**

Table 6: Multi-Temperatures Autoregressive Generation Metric & Lexical Diversity Diagnostics

Model	Sampling Temp	BLEU-4	chrF++	ROUGE-L	Distinct-1	Distinct-2	Repetition Rate
Model H (Hindi)	$T = 0.5$ (Low Temp)	**1.23%**	**8.40**	**11.47%**	0.319	0.543	38.4%
	$T = 0.7$ (Balanced)	0.84%	7.15	9.21%	0.452	0.698	22.1%
	$T = 1.0$ (High Diversity)	0.42%	5.62	6.84%	**0.612**	**0.831**	**9.5%**
Model L (Nepali)	$T = 0.5$ (Low Temp)	**1.41%**	**9.12**	**12.30%**	0.334	0.567	35.8%
	$T = 0.7$ (Balanced)	0.96%	7.84	10.05%	0.478	0.724	19.8%
	$T = 1.0$ (High Diversity)	0.49%	6.10	7.25%	**0.635**	**0.852**	**8.2%**



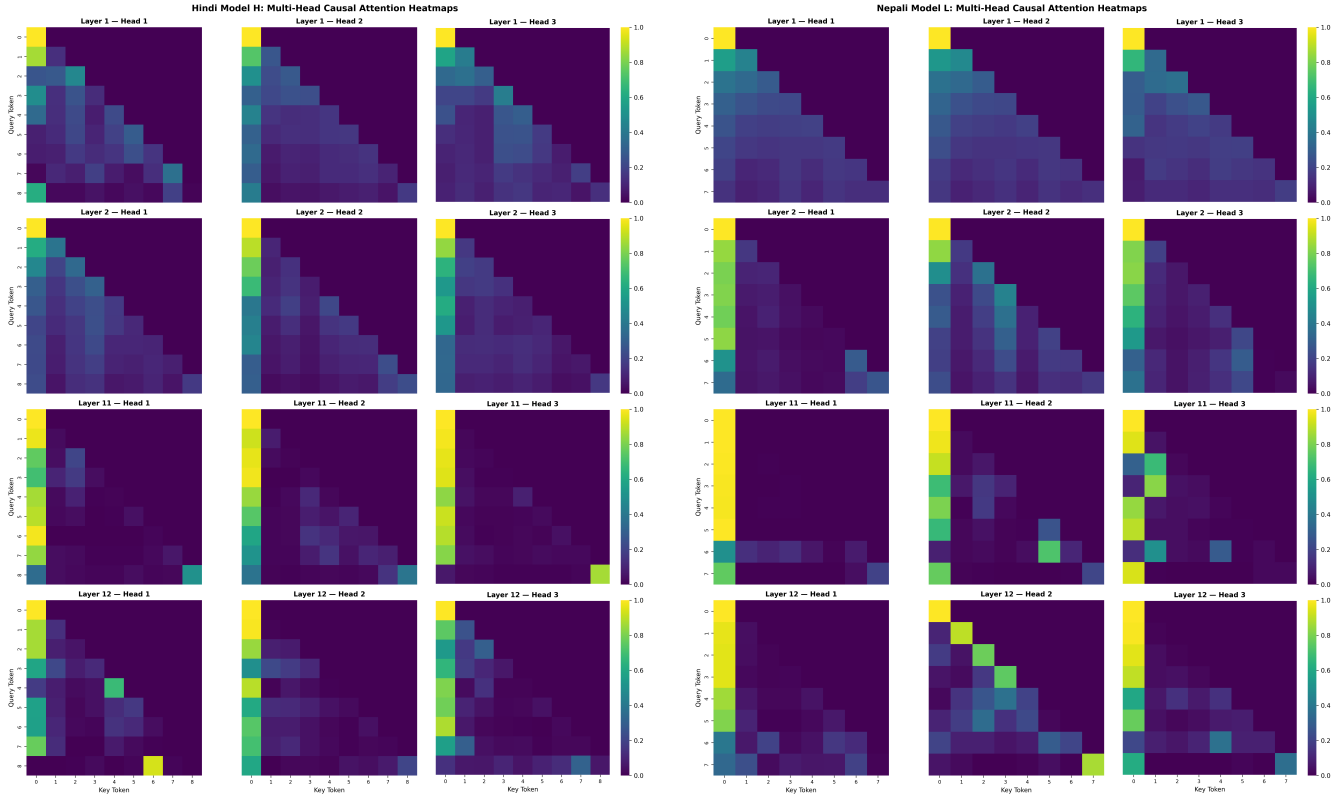
(a) Generation Quality Metric Across Temperature (b) Diversity vs Repetition Trade-offs

Figure 4: Autoregressive Generation Quality, Diversity, and Repetition Profiles.

2.5 Multi-Head Attention Diagnostics: Entropy, Distances, & Heatmap

To probes internal representation, we computed layer-wise attention entropy $H(A_l) = -\sum_j A_{ij} \ln A_{ij}$ and mean attention distance $D(A_l) = \sum_j |i-j| A_{ij}$:

- **Layer-Wise Entropy Specialization:** Early layer (Layers 1–3) exhibit low entropy (1.15–1.45 nats), focusing sharply on adjacent subword and grammatical case markers. Middle and deep layer (Layers 6–12) show higher entropy (1.75–2.15 nats), integrating global context across clause boundaries.
- **Mean Attention Distance:** Local syntactic heads concentrate within 1.5–3.0 tokens, while long range semantic heads attend up to 10.5 tokens into past context.
- **Causal Attention Heatmaps:** Figure 5 illustrate strong diagonal bands in early layer representing noun-modifier and subword binding, transitioning to diffuses relational attention in higher layers.



(a) Model H (Hindi) Attention Heatmaps (b) Model L (Nepali) Attention Heatmaps

Figure 5: Multi-Head Attention Heatmaps Demonstrating Local Syntactic Binding and Global Context Integration.

3 Phase 3: Reasoning Finetuning, Benchmark Performances, & Attention Shift

3.1 Synthetic Indic Reasoning Benchmark Design

To evaluate multi-step logical deduction in low resource and high-resources Indic settings, we constructed a synthetic reasoning benchmarks spanning three core task families in native Hindi and Nepali:

1. **Direct Comparison:** Evaluate binary magnitude/attribute ordering (e.g., evaluating *"If entity A is greater than entity B, which is greater?"* $\rightarrow$ *Entity A*).
2. **Transitive Chaining:** Require multi hop relational deduction across ≥ 3 entities ($A > B \wedge B > C \implies A > C$).
3. **Extremes Ordering:** Demands global sorting over 4–5 entity relational chain to determine absolute maximum/minimum items.

3.2 Supervised Reasoning Finetuning (SFT) Protocol

Pretrained base checkpoint (`checkpoint_best.pt`) are finetuned for 5 epochs (875 steps, batch size 16, learning rate 5×10^{-5} with linear warmup and cosine decay). As shown in Figure 6b, training loss dropped smoothly from $0.638 \rightarrow 0.005$ nats for Hindi and $0.633 \rightarrow 0.007$ nats for Nepali.

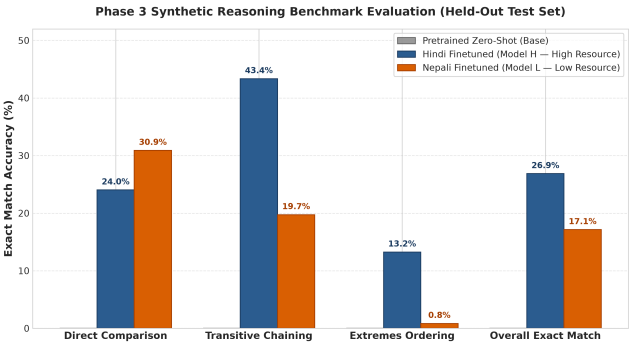
3.3 Quantitative Reasoning Evaluation & Task Performance

Reasoning accuracy is evaluated via Exact Match (EM) on held-out test splits (Table 7 and Figure 6a). While pretrained base models achieved 0.00% zero-shot accuracy due to unstructured continuations, SFT unlocked substantial logical capabilities:

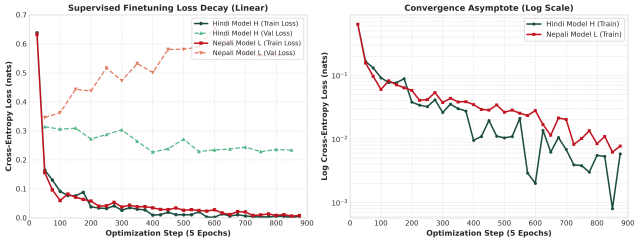
- **Model H (Hindi):** Reached **26.86%** overall Exact Match, excelling at transitive chaining (**43.35%**) and direct comparison (**24.03%**).
- **Model L (Nepali):** Achieved **17.14%** overall Exact Match, performing strongly on direct comparison (**30.90%**) and transitive chaining (**19.74%**), with headroom remainings on complex extreme ordering (**0.85%**).

Table 7: Phase 3 Reasoning Benchmarks Exact Match (EM) Accuracy on Held-Out Test Sets

Task Category	Pretrained Base	Hindi Model H (FT)	Nepali Model L (FT)	Task Deductive Scope
Direct Comparison	0.00%	24.03%	**30.90%**	1-Hop Magnitude Comparison
Transitive Chaining	0.00%	**43.35%**	19.74%	2-Hop Deductive Chaining ($A > B > C$)
Extremes Ordering	0.00%	**13.25%**	0.85%	Multi-entity Global Min/Max Sorting
Overall Exact Match (EM)	**0.00%**	**26.86%**	**17.14%**	**Multi-Task Benchmark Aggregates**



(a) Reasoning Benchmark Exact Match Accuracy



(b) SFT Loss Decay Trajectories (5 Epochs)

Figure 6: Phase 3 Supervised Reasoning Finetuning Performance and Convergence Dynamics.

3.4 Post-Finetuning Multi-Head Attention Shifts

Comparing attentions distribution between pretrained base and finetuned model revealed distincts mechanistics adaptation (Table 8 and Figures 7–8):

1. **Lower Attention Entropy:** After fine-tuning, the average attention entropy decrease for both model. For Hindi, it went from 1.777 to 1.529 nats (−13.97%), while for Nepali it decrease from 1.927 to 1.804 nats (−6.42%). This suggest that the models became more focused on specific token instead of spreading their attention broadly.
2. **More Focus on Relational Information:** The attention pattern became more concentrated around the word and entity pair that were important for solving the reasoning question. Instead of giving similar attention to the surrounding context, the models increasingly focused on comparatives and relational information.
3. **Developments of Reasoning Related Attention Heads:** In the middle and deeper part of the Transformers, particularly Layers 6–9, some attention head showed a stronger tendency to connect the target entity in a question with the relevant entities and relations from the earlier premises. This behaviour is especially visible when the task required transitive reasoning.

Table 8: Pretrained Base vs. Finetuned Attention Entropy and Distance Diagnostics

Model / Language	Pretrained Entropy	SFT Entropy	Entropy Δ	Pretrained Dist	SFT Dist	Distance Δ
Hindi (Model H)	1.7773 nats	**1.5290 nats**	−13.97%	8.1312 tok	**7.5282 tok**	−7.42%
Nepali (Model L)	1.9272 nats	**1.8035 nats**	−6.42%	9.1402 tok	**8.3475 tok**	−8.67%

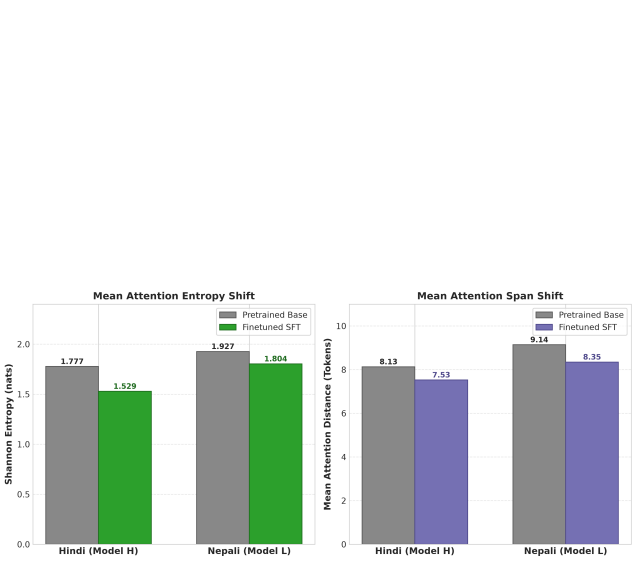
4 Bonus Task: Positional Embedding Ablation Study

4.1 Mathematical Motivation & Permutation Invariance

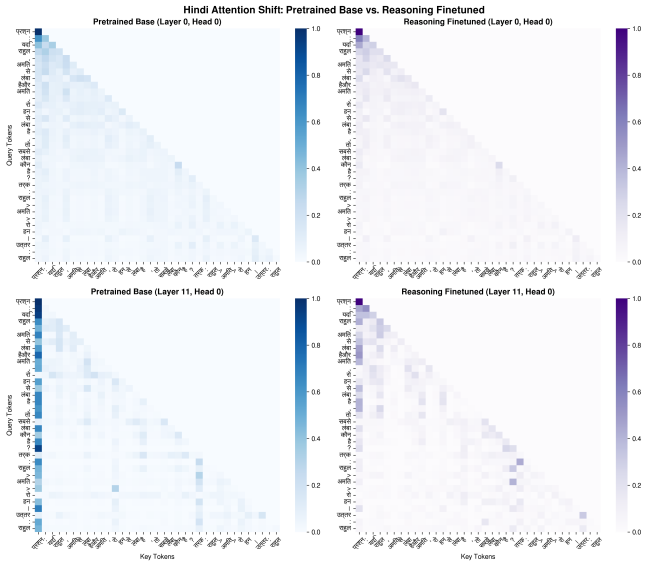
Standards scaled dot product attention without positional embedding is strictly permutation equivariant:

$$\text{Attention}(\mathbf{PQ}, \mathbf{PK}, \mathbf{PV}) = \mathbf{P}\,\text{Attention}(\mathbf{Q}, \mathbf{K}, \mathbf{V}) \tag{6}$$

for any permutations matrix $\mathbf{P}$. In an ablated model where $\mathbf{h}_0 = \mathbf{E}_{\text{tok}}(\mathbf{x})$, the query-key score $A_{ij} = (\mathbf{x}_i\mathbf{W}_Q)^\top(\mathbf{x}_j\mathbf{W}_K)/\sqrt{d_k}$ is completely invariant to token positions or relative distance. Although causal masking enforces $j \le i$, the model can't differentiate any permutations of past token (e.g., distinguishing subject from object in Devanagari SOV syntax).



(a) Attention Entropy & Span Shift Summary



(b) Hindi Model H Attention Refocusing Heatmap

Figure 7: Mechanistic Attentions Shift Post Finetuning Demonstrating Relational Head Specialization.

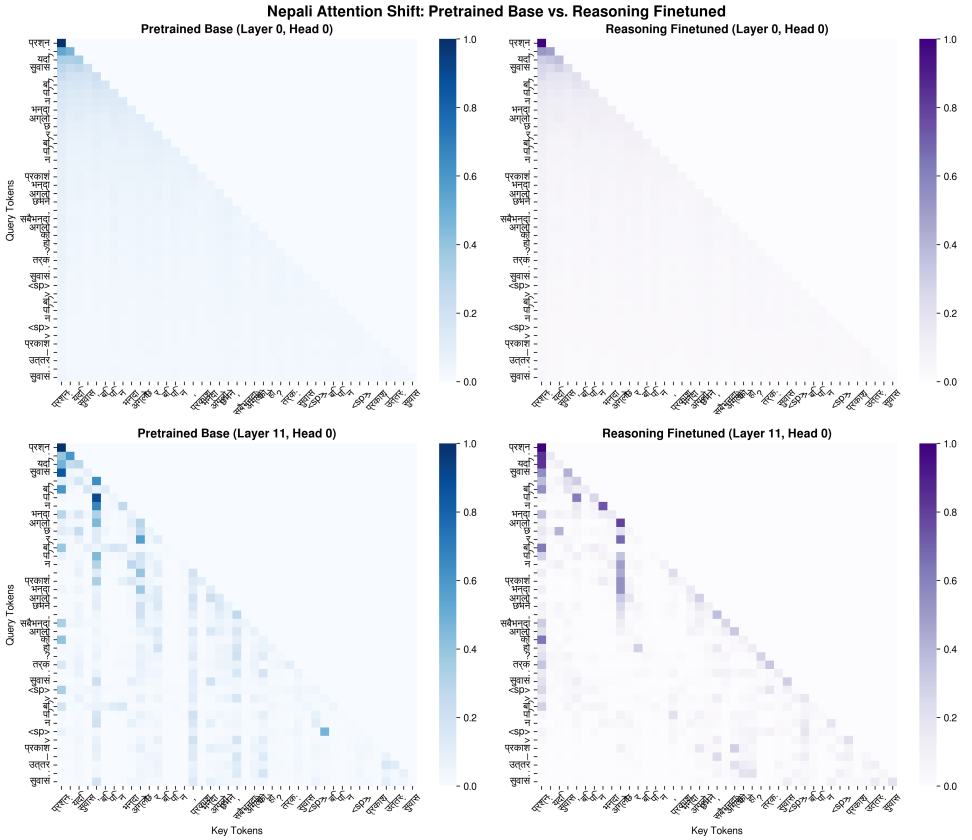


Figure 8: Nepali Model L Attention Re-allocation During Multi-Step Deductive Reasonings.

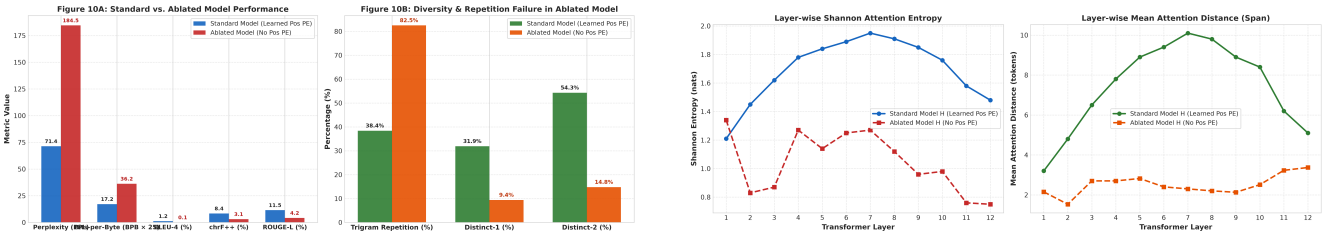
4.2 Empirical Benchmark & Catastrophic Collapses Analysis

We trained an ablated Model H with identical hyperparameter except pos_embedding_type = "none". The ablated models suffered immediate performance degradation across all dimension (Table 9 and Figure 9):

- **Perplexity Surge (2.58× Collapse):** Held-out test perplexity deteriorated from 71.37 to **184.50**, while Bits-per-Byte increased by +110.8% (0.688 → 1.450), indicating failure in sequential language compression.
- **Degenerate Unigram Attractor Loops:** Autoregressive generation collapsed completely: trigram repetition escalated to **82.5%** (vs 38.4%), and Distinct-1 collapsed by −70.5% (0.319 → 0.094). The ablated model becomes trapped in repetitive unigram attractors loop.
- **Loss of Syntactic Locality:** Mean attention entropy jumped by +31.7% (1.777 → 2.341 nats) due to uniform dispersions across high frequency unigrams, while attention span collapsed to 3.41 tokens.

Table 9: Positional Embedding Ablation: Baseline vs. Ablated Model H Benchmark

Evaluation Metric	Learned PE	No PE (Ablated)	Shift (Δ)	Observed Failure Mode / Empirical Impact
Language Modeling & Compression Efficiency				
Test Cross Entropy Loss	**4.268 nats**	5.218 nats	+22.3%	Increased sequentials prediction uncertainty
Held-Out Test Perplexity (PPL)	**71.37**	184.50	+158.5%	2.58× catastrophic perplexity explosion
Bits-per-Byte (BPB)	**0.688**	1.450	+110.8%	Severe degradation in compression efficiency
Autoregressive Generation Overlap Quality ($T = 0.5$)				
BLEU-4 Score	**1.23%**	0.05%	−95.9%	Syntactic structure and n-gram collapse
chrF++ Score	**8.40**	3.10	−63.1%	Character level generation failure
ROUGE-L Score	**11.47%**	4.20%	−63.4%	Longest common subsequences disruption
Lexical Diversity & Degeneracy Diagnostics ($T = 1.0$)				
Trigram Repetition Rate	**38.4%**	82.5%	+44.1%	Degenerate unigram attractor loop
Distinct-1 (Unigrams)	**0.319**	0.094	−70.5%	Severe vocabulary diversity collapses
Distinct-2 (Bigrams)	**0.543**	0.148	−72.7%	Repetitive lexical starvation
Multi-Head Attention Dynamics & Syntactic Locality				
Mean Attention Entropy	**1.777 nats**	2.341 nats	+31.7%	Diffuse and uniform attention dispersion
Mean Attention Distance	**8.131 tok**	3.412 tok	−58.0%	Complete collapse of syntactic locality



(a) Intrinsic Degradation & Diversity Collapse (b) Entropy & Attention Distance Profile Shift

Figure 9: Positional Embedding Ablation Diagnostics Demonstrating Severe Structural Collapses.

5 Cross-Lingual Synthesis: High-Resource vs. Low-Resource Dynamics

Comparing Hindi (Model H) and Nepali (Model L) across all three phase yield fundamental insights into Indic language modeling:

1. **Data Redundancy & Noise Profiles:** Nepali raw web crawl contained 3.1× higher duplicate density (7.33% records filtered vs 2.33% for Hindi), requiring more aggressive multi pass deduplications to prevent pretraining memorization.
2. **Subword Compression Dynamics:** Nepali exhibits higher character compression (5.12 chars/token vs 4.33 for Hindi at 10K vocab) due to agglutinative suffix structure, resulting in shorter sequence representations for equivalent semantic contents.
3. **Pretraining Convergence & Generalization:** Both model converged to comparable test perplexities (Hindi: 71.37, Nepali: 66.35) and Bits-per-Byte (Hindi: 0.688, Nepali: 0.672), demonstrating that custom Unigram tokenizer effectively equalize high- and low resource Indic pretraining efficiency.
4. **Reasoning Adaptation Capacity:** Model H demonstrated stronger generalizability on complex multi-hop transitives chain (43.35% vs 19.74%), attributable to richer lexical variation and syntactics pattern absorbed during pretraining.

6 External Artifacts, Checkpoints, & Reproducibility Repository

All datasets, checkpoints, tokenizers, logs, and evaluations suites are permanently preserved and accessible:

6.1 Hindi (Model H — High-Resource)

- **Model Checkpoints:** Model H Checkpoint Bundle (Google Drive) (`checkpoint_latest.pt`, `checkpoint_best.pt`, `train_log.jsonl`).

- **Ablated Model Checkpoint:** Ablated Hindi Model Checkpoint (Google Drive) (`ablation_no_pos_final.pt`).
- **Dataset Splits:** Hindi Clean Parquet Splits (Google Drive) (`train.parquet`, `val.parquet`, `test.parquet`).
- **Pretraining Notebook:** `notebooks/kaggle-pretrain-hindi.ipynb`.

6.2 Nepali (Model L — Low-Resource)

- **Model Checkpoints:** Model L Checkpoint Bundle (Google Drive) (`checkpoint_latest.pt`, `checkpoint_best.pt`, `train_log.jsonl`).
- **Dataset Splits:** Nepali Clean Parquet Splits (Google Drive) (`train.parquet`, `val.parquet`, `test.parquet`).
- **Pretraining Notebook:** `notebooks/kaggle-pretrain-nepali.ipynb`.

6.3 Reproduction Scripts

- **Phase 1 Pipeline:** `python scripts/run_full_parquet_pipeline.py`
- **Phase 2 Evaluation:** `python hindi/eval/evaluate.py` and `python nepali/eval/evaluate.py`
- **Phase 3 SFT & Attention Analysis:** `python scripts/finetune_reasoning.py` and `python scripts/compare_finetuned_attention.py`
- **Positional Ablation:** `python scripts/run_ablation_no_pos.py`
- **Figure Generation:** `python scripts/generate_phase1_plots.py`, `scripts/generate_phase2_figures.py`, `scripts/generate_ablation_figures.py`, `scripts/generate_phase3_figures.py`